BAYANIHANDR

Master Project Blueprint | Version 1.0

Terraform-Powered Disaster Recovery for Philippine Small Businesses

Repository: https://github.com/jervilangelo/bayanihan-dr

Planned public demo: https://jervilangelo.github.io/bayanihan-dr/

Development environment: Windows, VS Code, Claude Code

AWS region: ap-southeast-1 (Singapore)

Document status: Baseline plan; implementation and cost estimates are not verified results

Prepared: 04 November 2026

DOCUMENT CONTROL

Item	Policy
Purpose	Single source of truth for project scope, architecture, phases, controls, decisions, and acceptance criteria.
Change management	Update the blueprint and change log when architecture or scope changes. Preserve version history.
Source of truth	Keep an editable Markdown copy in docs/MASTER_BLUEPRINT.md; export this PDF as a readable snapshot.
Authority	Blueprint guides development, but never authorizes AWS charges, destructive actions, or publication without explicit approval.

1. Executive Summary

BayanihanDR is a low-cost disaster recovery proof of concept for a fictional agricultural cooperative in Naga City, Camarines Sur. The cooperative depends on a local Cooperative Management System (CMS). A typhoon, flood, or prolonged power failure can make the primary Linux server unavailable. BayanihanDR protects verified PostgreSQL backups off-site, provisions temporary AWS recovery infrastructure using Terraform, restores the CMS, validates its health, and records measurable recovery outcomes.

The project has two deliverables: (1) a real, reproducible technical disaster recovery workflow; and (2) a friendly, polished web dashboard that remains publicly accessible as a safe demonstration after AWS test resources are destroyed.

Success criteria

- A synthetic-data CMS works locally with PostgreSQL and Docker Compose.
- Backups are generated, checksummed, tested through restore, and securely stored off-site.
- Terraform can provision the approved AWS recovery environment reproducibly.
- A recovery drill restores the application and validates expected records.
- Actual RTO/RPO measurements and cost evidence are documented; targets are not presented as achieved until tested.
- A responsive GitHub Pages demo remains usable without a running AWS backend.
- Temporary billable resources are reviewed and removed after approved testing.

2. Business Case and Boundaries

Fictional customer

Bicol Farmers Cooperative (BFC), approximately 25 staff and 1,000 member farmers, uses an on-premises CMS to track membership, inventory, sales, contributions, and reports. All application records used in this project are synthetic.

Disaster scenario

Severe typhoon conditions cause flooding and an extended power outage. The primary office and local server become inaccessible. The recovery administrator operates from another trusted device and network; the original office must not be required to initiate recovery.

In scope

- Local CMS API, PostgreSQL schema/migrations, seed data, and basic health checks.
- Scheduled database backups, integrity checks, recovery-point inventory, retention, and restore verification.
- S3 storage with encryption, restricted access, and suitable retention/versioning decisions.
- Terraform modules for networking, compute, storage, IAM, and monitoring.
- A controlled recovery orchestrator and local operations dashboard.
- Recovery testing, CI/CD checks, GitHub Pages portfolio mode, runbooks, and cleanup.

Out of scope for v1

- Real farmer or financial data; production service-level guarantees; 24/7 hosted commercial operations.
- Multi-region active-active architecture, managed database clusters, payment processing, and complex multi-tenant authorization.
- Public execution of Terraform, public operational backend, and continuous paid AWS standby infrastructure.

3. Recovery Objectives and Definitions

Metric	Target / definition
RTO	30-minute target, measured from declared disaster/recovery start to verified CMS availability; document the exact stopwatch boundary.
RPO	1-hour target, measured as the time difference between the disaster event and the latest successfully restorable data point.
Backup frequency	Hourly as a design target only after scheduling, verification, and end-to-end restore prove it.
Evidence	Timestamped backup manifests, logs, restore checks, health checks, recorded elapsed time, and AWS usage/cost records.

A successful restore from a backup does not by itself prove a 1-hour RPO. Scheduling, backup completion, verification, clock consistency, and disaster timing all matter. Test missed or corrupt backups and report limitations.

4. System Architecture

Primary environment

A local Docker Compose stack runs the CMS API and PostgreSQL. A backup job produces a consistent database dump and metadata manifest, verifies checksums, and periodically performs test restores. Backup copies are uploaded securely to S3.

Recovery environment

A trusted recovery operator uses a separately available workstation with AWS access and a protected Terraform state. Terraform provisions an isolated AWS network and a temporary EC2 Linux host. Bootstrap scripts configure Docker or application dependencies, retrieve the approved backup, restore PostgreSQL, start the CMS, and perform health and record checks.

Dashboard and backend separation

A React/Vite dashboard has a local operational mode connected to an authenticated FastAPI recovery backend and a public GitHub Pages mode backed by sanitized static JSON. Public mode is read-only or simulation-only and must never call real recovery endpoints.

Conceptual data flow

Local CMS + PostgreSQL -> scheduled backup + checksum + restore test -> encrypted S3 backup -> approved recovery orchestration -> Terraform EC2 environment -> restore and verify -> evidence and metrics -> sanitized public demo.

Key failure-domain requirement

Backup copies, recovery credentials, Terraform state, and recovery instructions must remain usable if the primary office and server are lost. A local-only orchestration service is a convenience, not the sole recovery path.

5. Technology Decisions

Area	Preferred choice	Reason
Frontend	React, Vite, TypeScript, Tailwind	Static GitHub Pages deployment, typed interfaces, low operating cost.
Motion	Motion for React	Smooth restrained transitions; reduced-motion support.
UI	TasteSkill, UI/UX Pro Max, selective shadcn/ui, Lucide	Distinctive visual system without generic template appearance.
Charts	Recharts when needed	Useful backup/recovery trends, not decorative analytics.
CMS	FastAPI + PostgreSQL + Docker Compose	Practical Linux/database recovery exercise.
Operations API	Python FastAPI + optional SQLite metadata	Controlled local jobs, audit trail, API contracts.
IaC	Terraform + official HashiCorp skills	Reusable, reviewable infrastructure and portfolio relevance.
AWS	EC2, VPC, S3, IAM, EBS, CloudWatch, optional SSM	Low-complexity cold recovery infrastructure.
CI/CD	GitHub Actions + Playwright + pytest	Repeatable tests and static frontend publication.

6. Dashboard Product Specification

Audience: nontechnical small-business administrators and technical recovery operators. Design principle: a clear answer to "Are we protected?" and "What should we do next?" without drowning the user in AWS terminology.

Visual language

- Default light theme; warm off-white #F8FAF9, primary sage #397D6A, pale green #DCEDE7, soft blue #7DA8C7, charcoal #263B36, warning gold #D9A441.
- Custom layout and information hierarchy; restrained shadows, clear typography, whitespace, meaningful status labels.
- Avoid repetitive KPI cards, gradients, clutter, flashy dashboards, decorative charts, and dark cyberpunk styling.
- Motion for React: typical 150-300 ms transitions, gentle content entrances, loading skeletons, meaningful progress steps, and prefers-reduced-motion support.
- Accessible contrast, keyboard navigation, focus indicators, responsive layout, and readable failure states.

Five core pages

Page	User-facing capabilities
Overview	Primary status, last verified backup, recovery readiness, target RTO, recent activity, clear next action.
Backups	Backup list, size/time, verification, checksums/metadata summary, selectable recovery points.
Recovery	Preflight checklist, recovery point, plan preview, explicit approval, step-by-step progress, verification, result.
Activity	Chronological backup, infrastructure, recovery, error and audit events with filters.
Settings	Recovery targets, safe preferences, environment/region information; privileged configuration only in local mode.

Two modes

Public demo: static, sanitized recorded test evidence and clearly labeled simulation; no privileged actions or false live status. **Local operational:** authenticated backend with authorization, explicit approval, audit logs, and bounded operations. Use a typed provider interface to swap mock and real data without rewriting pages.

7. Backup, Restore and Recovery Workflow

Backup pipeline

- Create a consistent PostgreSQL backup using pg_dump or equivalent supported procedure.
- Generate a manifest: backup ID, UTC timestamp, source version, schema/app version, size, checksum, verification state.
- Validate the archive and periodically restore into a disposable PostgreSQL instance; checksum alone does not prove restorable data.
- Encrypt backup objects in S3, enforce least-privilege access, set retention and deletion safeguards.
- Track schedule failures, upload failures, missing backups, and stale verification; do not mark unverified backups as protected.

Recovery sequence

- Declare incident and confirm primary unavailability; start RTO timer.
- Confirm independent access to backup storage, AWS credentials, Terraform state, and runbook.
- Select latest verified compatible recovery point and review plan and expected cost.
- Approve job explicitly; block duplicate concurrent jobs.
- Provision approved recovery infrastructure; bootstrap application environment.
- Download and verify selected backup; restore database and start CMS.
- Run health checks and verify known records and expected application behavior.
- Record RTO, RPO, timestamps, outcomes, logs, and any exceptions.
- Preserve evidence and obtain separate approval for teardown.

Failure handling

Fail closed if backup integrity fails, authentication is missing, state is locked, a job is already running, or estimated actions differ from the reviewed plan. Preserve diagnostics and provide safe retry guidance.

8. Terraform and AWS Architecture

Terraform module boundaries

Module	Resources / concerns
networking	VPC, public subnet, Internet Gateway, route table, restrictive security groups.
compute	Small temporary EC2 Linux instance, user data/bootstrap, EBS volume, instance profile.
storage	Encrypted S3 backup bucket, versioning/retention decisions, restricted bucket policy.
identity	Least-privilege IAM roles and policies; no long-lived credentials in source.
monitoring	CloudWatch log groups and minimal operational signals; bounded retention.

State and bootstrap

Use protected remote Terraform state with locking supported by the selected backend and tooling version. The state backend must exist independently of the primary site and be created through an explicit, separately reviewed bootstrap process. Never commit state or secrets. Document bootstrap and disaster access.

Network and access choices

Prefer SSM Session Manager if practical; otherwise tightly restrict SSH to an approved IP and use short-lived credentials. No public database exposure. A public IPv4 address, EBS, S3 requests, logs, and data transfer may incur costs. No NAT Gateway, RDS, ALB, EKS, or always-on paid services in v1 unless separately approved.

Validation and review

Use terraform fmt, validate, plan, lint/security scanning where available, provider version constraints, variables, outputs, tags, and documented module interfaces. Never run terraform apply or destroy without explicit user approval.

9. Security, Privacy and Governance

- Use synthetic cooperative data only; no real personal, payroll, membership, or financial records.
- Never commit .env credentials, access keys, private keys, backup contents, Terraform state, or privileged endpoint URLs.
- Use short-lived/role-based AWS authentication where possible, least privilege, encryption in transit and at rest, and explicit credential rotation procedures.
- Separate public demo assets from operational data; sanitize logs, account IDs, IPs, secrets, and internal paths before publishing.
- Local recovery operations require authenticated administrator access, authorization checks, CSRF/origin protections where relevant, allowlisted job types, and auditable approvals.
- Do not accept arbitrary shell commands or Terraform arguments from browser input.
- Keep operations available from a trusted alternate workstation after primary-site loss.
- Record resource creation, approval, recovery, and destruction in an audit trail.

10. AWS Cost Model and Controls

Target total test spending: PHP 300-500. This is a planning budget, not a hard cap or verified quote. Recalculate against current ap-southeast-1 pricing and actual instance runtime before any apply.

Cost driver	Control
EC2 runtime	Choose a small compatible instance, run only during approved recovery drills, stop/terminate promptly.
EBS	Keep volumes minimal and verify cleanup of detached volumes/snapshots.
Public IPv4	Include hourly address cost if assigned; avoid unused elastic IPs.
S3	Small synthetic backups; lifecycle/retention and versioning reviewed carefully.
CloudWatch	Short log retention; avoid high-frequency custom metrics.
Data transfer / requests	Estimate backup uploads, downloads, API calls, and cross-region traffic.
State storage	Include any ongoing state backend storage/locking cost after teardown.

Configure AWS billing alerts before deployment, but explain that alerts do not stop charges. Record baseline resource inventory, pre-deployment plan, runtime duration, and post-destroy inventory.

11. Repository and GitHub Pages

Repository: https://github.com/jervilangelo/bayanihan-dr

Planned public URL: https://jervilangelo.github.io/bayanihan-dr/

Vite base: /bayanihan-dr/ . Use a GitHub Pages-compatible routing strategy and verify direct navigation and refresh.

Proposed layout

frontend/ - public dashboard and provider interfaces; backend/ - local recovery API; cooperative-system/ - CMS and database; terraform/modules/ and terraform/environments/recovery/; scripts/backup, restore, validation; tests/; docs/; .github/workflows/; README.md; .gitignore.

GitHub workflows

- Frontend lint/typecheck/build and browser tests; publish static artifacts to GitHub Pages after approval.
- Backend pytest and static checks; Terraform fmt, validate, and security checks.
- No automatic Terraform apply or destroy on ordinary pushes.
- Use least-privilege GitHub Actions permissions; review deployment environments and branch protections.
- Never push or publish without explicit approval.

12. Development Phases and Gates

Phase 0 - Blueprint and environment

Build: Create authoritative Markdown blueprint + PDF snapshot; inspect tools, skills, repository and architecture; establish acceptance criteria.

Exit gate: Blueprint saved, reviewed, and versioned; no AWS resources.

Approval: Stop for user approval before moving to the next phase.

Phase 1 - Local cooperative CMS

Build: Docker Compose, FastAPI, PostgreSQL, migrations, synthetic farmer/inventory/transaction data, CRUD and health endpoints.

Exit gate: Local app runs; API and DB tests pass.

Approval: Stop for user approval before moving to the next phase.

Phase 2 - Dashboard and animations

Build: React/Vite pages, distinctive UI system, mock provider, recovery simulation, accessibility and Playwright checks.

Exit gate: Five pages function; simulation clearly labeled; build/tests pass.

Approval: Stop for user approval before moving to the next phase.

Phase 3 - Backup and restore

Build: Scheduled pg_dump, metadata, checksums, retention, local restore verification, failure handling.

Exit gate: Repeatable successful restore; bad backup detection.

Approval: Stop for user approval before moving to the next phase.

Phase 4 - Terraform infrastructure

Build: Modules for networking, compute, S3, IAM, CloudWatch; remote state plan; fmt/validate/plan.

Exit gate: Reviewed, valid plan; no apply yet.

Approval: Stop for user approval before moving to the next phase.

Phase 5 - Approved AWS integration

Build: Cost review, billing alerts, explicit apply approval, deploy, S3 backup upload, verify access.

Exit gate: Resource inventory and verified AWS backups.

Approval: Stop for user approval before moving to the next phase.

Phase 6 - Recovery orchestration

Build: Authenticated FastAPI job runner, bounded Terraform workflow, backup selection, restore and checks.

Exit gate: Controlled recovery job succeeds with audit evidence.

Approval: Stop for user approval before moving to the next phase.

Phase 7 - Disaster drill and metrics

Build: Stop primary CMS, execute restore from trusted alternate context, verify data, measure RTO/RPO and cost.

Exit gate: Timestamped evidence and honest pass/fail report.

Approval: Stop for user approval before moving to the next phase.

Phase 8 - CI/CD and quality

Build: GitHub Actions, Python/TypeScript/Terraform checks, Playwright, security checks.

Exit gate: Pipelines pass without unapproved AWS provisioning.

Approval: Stop for user approval before moving to the next phase.

Phase 9 - GitHub Pages publication

Build: Publish safe static demo with sanitized recorded results and compatible routing.

Exit gate: Public site works while AWS is offline.

Approval: Stop for user approval before moving to the next phase.

Phase 10 - Documentation and teardown

Build: README, diagrams, runbook, cost report, screenshots, lessons, destroy review and cleanup.

Exit gate: Docs complete, public demo persists, residual charges checked.

Approval: Stop for user approval before moving to the next phase.

13. Skill-to-Phase Mapping

Skill	Use
HashiCorp Terraform plugin	Phases 0, 4, 5, 6, 8, 10: module conventions, state, testing, lifecycle.
Superpowers	All phases: architecture, decomposition, test-first work, debugging, review.
TasteSkill	Phases 2 and 9: custom visual language, composition, interaction polish.
UI/UX Pro Max	Phases 2 and 9: usability, accessibility, responsive patterns.
Playwright CLI Skill	Phases 2, 7, 8, 9: browser behavior, responsive and visual checks.

Claude Code must verify each installed skill and read its actual instructions before claiming to use it. Missing or incompatible skills should be reported, not invented.

14. Test Strategy and Acceptance

Unit and integration

- CMS CRUD, validation, migrations, and health endpoint tests.
- Backup script error handling, manifest/checksum generation, and restore verification tests.
- Recovery job state machine, authorization, concurrency and retry behavior tests.
- Frontend provider contracts, navigation, loading/error states, and demo simulation tests.

End-to-end and operations

- Playwright across desktop/mobile sizes and reduced-motion setting.
- Terraform formatting, validation, reviewable plan, and security checks.
- Controlled disaster drill with known synthetic records and timestamped verification.
- Public site verification after AWS resources are destroyed.

Evidence

Store sanitized screenshots, logs, test outputs, measured durations, backup manifest examples, infrastructure diagrams, and cost observations. Never fabricate data or claim targets were met without proof.

15. Risk Register

Risk	Mitigation
Unexpected AWS charges	Preflight estimates, short-lived resources, billing alerts, explicit apply/destroy approval, post-test inventory.
Primary site loss blocks recovery	Independent backup access, remote state, runbook, alternate trusted workstation.
Unrestorable backups	Scheduled disposable restore tests, checksums, manifests, compatible versions.
Secrets exposed publicly	Separate builds, sanitized static data, secret scanning, no public privileged APIs.
Insecure recovery automation	Authentication, allowlisted actions, plan review, job locks, audit trail.
Overbuilt UI delays infrastructure	Five-page scope, purposeful animations, functional demo first.
RTO/RPO missed	Measure honestly, isolate slow steps, improve and rerun; no fabricated compliance.
Windows/toolchain incompatibility	Inspect versions, Docker availability, long-path support, document workarounds.

16. Definition of Done and Change Control

- All required code is committed in the authorized repository, with no secrets.
- CMS, backups, Terraform deployment, and recovery drill are reproducible from documentation.
- Measured recovery performance, failure cases, and actual AWS costs are recorded.
- Dashboard works as both safe static portfolio demo and controlled local operations UI.
- Automated tests and accessibility checks pass or limitations are documented.
- AWS resources are destroyed only after explicit approval, with residual charges reviewed.
- Blueprint is updated for every material architecture, scope, cost, or workflow change.

Change log

Version	Date	Change
1.0	2026-11-04	Initial comprehensive blueprint; proposed architecture and phased delivery.

Open decisions for Phase 0

- Confirm whether to use HashRouter or static route fallback for GitHub Pages.
- Select Terraform remote-state backend and locking strategy; document bootstrap and residual costs.
- Choose recovery access method (SSM preferred where practical; otherwise restricted SSH).
- Determine backup schedule/retention, encryption key management, and restore-test cadence.
- Decide exact local authentication method and how alternate-workstation recovery is initiated.
- Verify actual current AWS pricing and the final test budget before provisioning.